

Parcial de Python — v5

Condicionales · for/while · def & lambda · Ciencia de Datos (Nivel experto)

🕒 120 min
📄 20 preguntas
🎯 10 puntos

Instrucciones generales

§ Individual, en computador

§ Solo Python estándar

§ Guarde cada respuesta en un .py

§ Sin conexión a internet

§ Cada pregunta vale **0.5 pts**

§ Se evalúa lógica, estilo y casos borde

λ = uso obligatorio de **lambda** | = manejo de **casos borde** | = solución en **una sola pasada (O(n))**

| = al menos **dos lambdas anidadas**

I CONDICIONALES · PREGUNTAS 1 – 5

- 1 `diagnosticar_matriz_confusion(tp, tn, fp, fn)` → calcula precision/recall y clasifica en Excelente/Bueno/Regular/Pobre. Maneje "sin datos" y "clase ausente".
- 2 Con `valores=[5,8,12,3,7,9,15,6,11,4]`, cuente valores por debajo del 25 %, entre 25–75 % y por encima del 75 % del máximo.
- 3 `clasificar_missing(valor)` → "NaN", "nulo", "vacío", "cero" o "válido".
- 4 De un `pipeline` de etapas, imprima la primera con estado `"error"` o "Pipeline completo"; mencione pendientes.
- 5 `decidir_modelo(a, b, criterio)` → "A"/"B"/"empate". Valide criterio existente.

II CICLOS FOR · PREGUNTAS 6 – 10

- 6 Cuente pares de elementos que suman 10. Optimice con `set` (evita doble bucle).
- 7 De una lista de eventos con timestamp, calcule intervalos entre clicks consecutivos y su promedio.
- 8 Máxima ganancia tipo "buy/sell": con una sola pasada, halle `max(lista[j] - lista[i])` con $i < j$.
- 9 Frecuencia de palabras + lista ordenada por freq desc y alfabético asc sin `sorted()` (selección).
- 10 Con `for` anidados, en una sola pasada calcule diagonales y total de una matriz 3×3 sin `sum()`.

III CICLOS WHILE · PREGUNTAS 11 – 15

- 11 Búsqueda binaria con `while`; retorne índice o -1.
- 12 MCD y MCM con Euclides para 48 y 180.
- 13 Perceptrón simple con regla de aprendizaje: cuente épocas hasta convergencia (< 100).
- 14 Cola de prioridad con `while` sin `heapq`: extraiga siempre la menor prioridad.
- 15 Juego del caos (logística $x = 3.9x(1-x)$): 10000 iteraciones, estime proporción en [0.4, 0.6] tras el transitorio.

- 16 `calcular_metricas` con `def` y con `lambda` (dos lambdas anidadas). Accuracy, precision, recall, F1.
- 17 Solo con `lambda` + `sorted` + `filter` : nombres de empleados activos de IT ordenados por salario desc.
- 18 Una comprensión con dos lambdas: pares→cuadrado, impares→cubo sobre `[1..10]` .
- 19 Producto de primos en [2,50] usando solo `lambda` , `filter` y `reduce` (sin `def`).
- 20 `crear_pipeline(*funcs)` que retorne una `lambda` con composición funcional. Sin args → identidad.

Rúbrica y Soluciones — v5

Documento para el docente · uso interno

 Confidencial



RÚBRICA DE EVALUACIÓN

CRITERIO	PUNTOS
Sintaxis correcta y uso adecuado de estructuras	0.2
Lógica correcta del algoritmo	0.2
Manejo de casos borde (listas vacías, división por cero, NaN)	0.1
Total por pregunta	0.5

Nota sobre badges: ⚡ una sola pasada $O(n)$; dos o más pasadas = máximo 0.3 pts. · 🦋 menos de dos lambda = máximo 0.3 pts. · ⚙ sin manejo de casos borde = máximo 0.4 pts.



SOLUCIONES BREVES (REFERENCIA)

```
# 1
def diagnosticar_matriz_confusion(tp, tn, fp, fn):
    if tp+tn+fp+fn == 0: return "sin datos"
    if tp+fn == 0 or tn+fp == 0: return "clase ausente"
    p = tp/(tp+fp) if (tp+fp) else 0
    r = tn/(tn+fn) if (tn+fn) else 0
    if p>=0.9 and r>=0.9: return "Excelente"
    if p>=0.7 and r>=0.7: return "Bueno"
    if p>=0.5 and r>=0.5: return "Regular"
    return "Pobre"

# 3
def clasificar_missing(v):
    if v != v: return "NaN"
    if v is None: return "nulo"
    if v == "" or v == []: return "vacío"
    if v == 0 and not isinstance(v, bool): return "cero"
    return "válido"

# 6 — una pasada con set
datos = [1,9,2,8,3,7,4,6,5,1,9]
vistos, pares = set(), 0
for x in datos:
    if (10 - x) in vistos: pares += 1
    vistos.add(x)

# 8 — una pasada
precios = [7,1,5,3,6,4,9,2,8]
min_p = precios[0]; mejor = 0; par = (0,0)
for j in range(1, len(precios)):
    g = precios[j] - min_p
    if g > mejor: mejor, par = g, (precios.index(min_p), j)
    if precios[j] < min_p: min_p = precios[j]

# 9 — ordenamiento por selección
corpus = ["data", "science", "python", "data", "ai", "python", "data"]
fr = {}
for p in corpus: fr[p] = fr.get(p, 0) + 1
items = list(fr.items()); n = len(items)
for i in range(n):
    k = i
    for j in range(i+1, n):
        if items[j][1] > items[k][1] or \
```

```
(items[j][1] == items[k][1] and items[j][0] < items[k][0]):
        k = j
    items[i], items[k] = items[k], items[i]

# 10 — una pasada
m = [[1,2,3],[4,5,6],[7,8,9]]
d1 = d2 = tot = 0; n = len(m)
for i in range(n):
    for j in range(n):
        tot += m[i][j]
        if i == j: d1 += m[i][j]
        if i + j == n-1: d2 += m[i][j]

# 11
def bb(l, o):
    if not l: return -1
    i, d = 0, len(l)-1
    while i <= d:
        m = (i+d)//2
        if l[m] == o: return m
        elif l[m] < o: i = m+1
        else: d = m-1
    return -1

# 13 — perceptrón
X = [(1,1,1),(1,0,0),(0,1,0),(0,0,0)]
y = [1,0,0,0]
w = [0.0,0.0]; b = 0.0; lr = 0.1; ep = 0
while ep < 100:
    errs = 0
    for xi, yi in zip(X, y):
        pred = 1 if w[0]*xi[0]+w[1]*xi[1]+b > 0 else 0
        e = yi - pred
        if e:
            w[0] += lr*e*xi[0]
            w[1] += lr*e*xi[1]
            b += lr*e; errs += 1
    ep += 1
    if errs == 0: break

# 14 — cola de prioridad
tareas = [("t1",5),("t2",1),("t3",3),("t4",1),("t5",4)]
res = []
while tareas:
```

```

k = 0
for i in range(1, len(tareas)):
    if tareas[i][1] < tareas[k][1]: k = i
res.append(tareas.pop(k))

# 15 - juego del caos
x = 0.5; N = 10000; d = 0
for i in range(N):
    x = 3.9*x*(1-x)
    if i >= 100 and 0.4 <= x <= 0.6: d += 1
prop = d / (N - 100)

# 17 - solo Lambdas
emps = [
    {"nombre": "Ana", "depto": "IT", "salario": 4500, "activo": True},
    {"nombre": "Luis", "depto": "HR", "salario": 3200, "activo": False},
    {"nombre": "Marta", "depto": "IT", "salario": 5100, "activo": True},
    {"nombre": "Pedro", "depto": "HR", "salario": 2800, "activo": True},
    {"nombre": "Sofia", "depto": "IT", "salario": 4800, "activo": True},
]
res = list(map(lambda e: e["nombre"],
    sorted(
        filter(lambda e: e["activo"] and e["depto"]=="IT", emps),
        key=lambda e: -e["salario"])))

# 18 - una comprensión, dos Lambdas
nums = [1,2,3,4,5,6,7,8,9,10]
res = [(lambda par: (lambda x: x*x) if par else (lambda x: x**3)))(n)
    for n in nums]

# 19 - primos con Lambda
from functools import reduce
es_primo = lambda n: n > 1 and all(n % i != 0
    for i in range(2, int(n**0.5) + 1))
primos = filter(es_primo, range(2, 51))
prod = reduce(lambda a, b: a*b, primos)

# 20 - pipeline funcional
def crear_pipeline(*tf):
    if not tf: return lambda x: x
    return lambda x: reduce(lambda acc, f: f(acc), tf, x)

doblar = lambda x: x * 2
sumar_10 = lambda x: x + 10
cuadrar = lambda x: x ** 2
pipe = crear_pipeline(doblar, sumar_10, cuadrar)
print(pipe(3)) # 256

```